សប្តាហ៍ទី 6 · Objects និង lifecycle

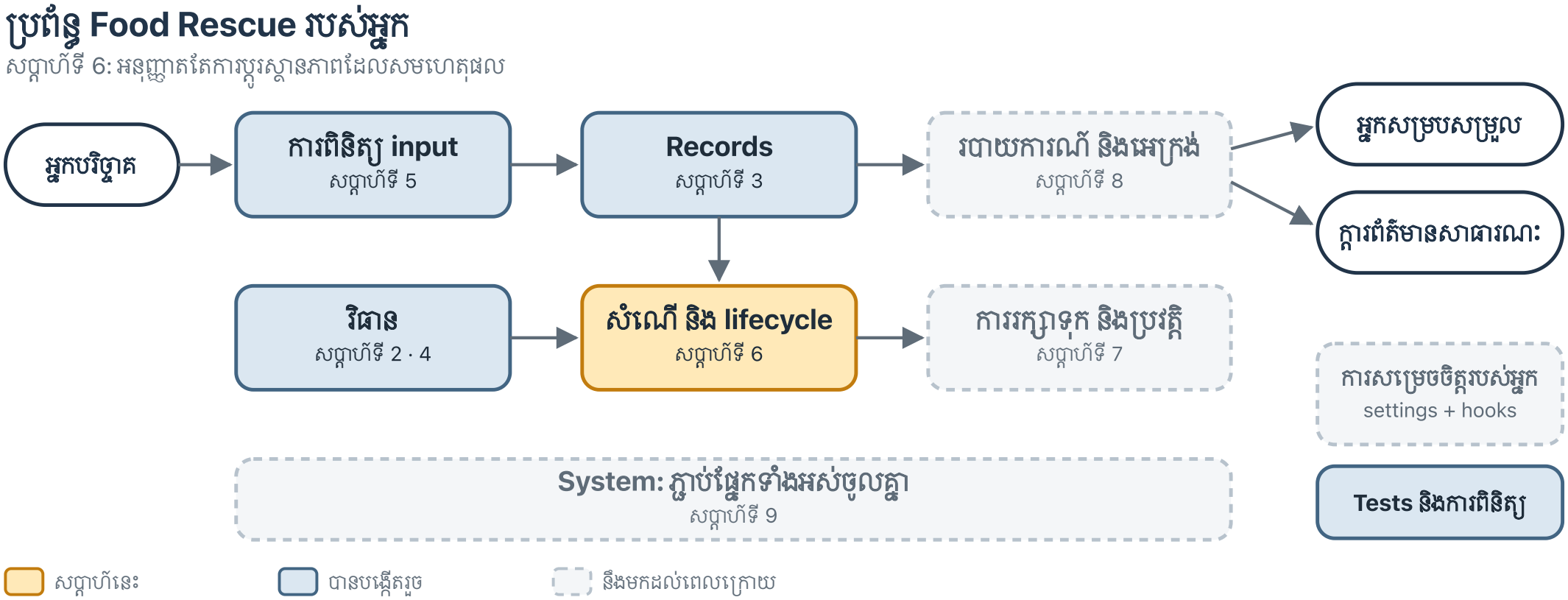
Programming for Information Systems

ធ្វើឱ្យស្ថានភាពខុស កើតឡើងបានពិបាក

សប្តាហ៍នេះ

សំណួរ	តើប្រព័ន្ធរក្សារបស់នីមួយៗឱ្យនៅក្នុងស្ថានភាពសមហេតុផល ដោយរបៀបណា?
Python	ការអាន class, object, attribute, method, `self`; dictionary នៃការប្តូរដែលបានអនុញ្ញាត
គោលការណ៍ទី 6	ធ្វើឱ្យស្ថានភាពខុស កើតឡើងបានពិបាក
អ្នកបង្កើត	`can_move(old_status, new_status)`

ប្រព័ន្ធរបស់អ្នករហូតមកដល់ពេលនេះ



គំនិតសំខាន់: entity, status, workflow

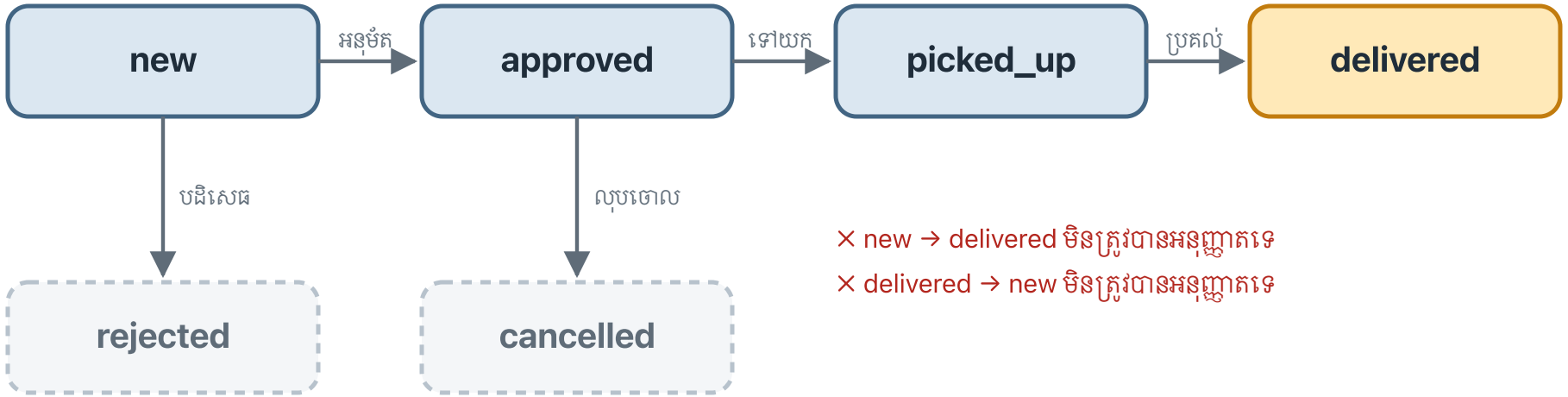
- **entity** គឺជារបស់ពិតមួយដែលប្រព័ន្ធតាមដាន: កញ្ចប់អីវ៉ាន់ និស្សិត សំណើ។
- **status** របស់វាប្រាប់ថា វានៅត្រង់ណាក្នុងដំណើររបស់វា។
- **workflow** គឺជាសំណុំនៃការប្តូរដែលសមហេតុផល។

> កញ្ចប់អីវ៉ាន់មិនអាច *បានប្រគល់* មុនពេល *បានផ្ញើចេញ* ទេ។ កញ្ចប់ដែលបានប្រគល់ មិនត្រឡប់ទៅជា *បានកុម្ម៉ង់* វិញទេ។

ព្រួញនីមួយៗ គឺជាការប្តូរដែលបានអនុញ្ញាត

lifecycle របស់សំណើ

អនុញ្ញាតតែការប្តូរទាំងនេះប៉ុណ្ណោះ។ ក្រៅពីនេះត្រូវបានបដិសេធ។



class គឺជាប្លង់

```python
class Parcel:
    def __init__(self, code, address):
        self.code = code
        self.address = address
        self.status = "ordered"

    def describe(self):
        return f"{self.code} to {self.address}: {self.status}"

parcel_a = Parcel("P01", "Battambang")
print(parcel_a.describe())        # P01 to Battambang: ordered
```

`__init__`, `self`, attribute, method

- `__init__` run ពេលអ្នកបង្កើត object។ វាកំណត់ value **ចាប់ផ្តើម**។
- `self` មានន័យថា **object នេះ**: `self.status` គឺជា status របស់កញ្ចប់នេះ។
- **attribute** គឺជា value នៅលើ object ម្ចាស់: `parcel_a.status`។
- **method** គឺជា function នៅក្នុង class: `parcel_a.describe()`។

```
parcel_a.status = "shipped"
print(parcel_a.describe())        # P01 to Battambang: shipped
print(parcel_b.describe())        # P02 to Siem Reap: ordered
```

អាន class ពិតមួយ: `fr.Offer`

```
class Offer:
    def __init__(self, offer_id, donor, food, category, portions, hours_left, contact=""):
        self.id = offer_id
        ...
        self.status = "new"
        self.partner = ""

    def move_to(self, new_status, is_allowed):
        if not is_allowed(self.status, new_status):
            return False
        self.status = new_status
        return True
```

Python មិនឃាត់អ្នកទេ

```
offer = fr.Offer("F101", "Sunrise Bakery", "Bread rolls", "bread", 30, 5)
offer.status = "deliverd"          # វាយខុស ហើយរំលងគ្រប់ជំហាន
print(offer.is_open())             # False
```

- មិន open: វាបាត់ពីបញ្ជីរបស់អ្នកសម្របសម្រួល។
- មិនមែន "delivered": វាក៏មិនត្រូវបានរាប់ជាបានប្រគល់ដែរ។

> Python មិនអាចធ្វើឱ្យស្ថានភាពខុស **មិនអាចកើតឡើងបាន** ទេ។ ការរចនាល្អធ្វើឱ្យវា **កើតឡើងបានពិបាក**។

dictionary នៃការប្តូរដែលបានអនុញ្ញាត

```
NEXT_LIGHT = {"green": ["yellow"], "yellow": ["red"], "red": ["green"]}
print("yellow" in NEXT_LIGHT["green"])     # True
print("red" in NEXT_LIGHT["green"])        # False

print(fr.ALLOWED_MOVES["approved"])        # ['picked_up', 'cancelled']
print(fr.ALLOWED_MOVES["delivered"])       # []  end state មួយ
```

- key នីមួយៗជា status ហើយ value នីមួយៗជា **list** នៃ status បន្ទាប់។
- workflow គឺជា **ទិន្នន័យ**: ប្តូរ dictionary មិនមែនប្តូរ logic ទេ។

`move_to` ស្ងួរវិធានជាមុនសិន

```
curry = fr.Offer("F102", "Lotus Cafe", "Vegetable curry", "rice meals", 12, 4)
print(curry.move_to("delivered", fr.models.can_move), curry.status)  # False new
print(curry.move_to("approved", fr.models.can_move), curry.status)   # True approved
```

- វិធានគឺជា function ដែលបញ្ជូនជា value (ដូច hook និង `fr.check`)។
- ការប្តូរដែលត្រូវបានបដិសេធ **មិនប្តូរអ្វីទាំងអស់** ហើយ return `False`។
- វិធាននៅ **ជាប់នឹងទិន្នន័យ** ដែលវាការពារ។

លំហាត់: class ផ្ទាល់ខ្លួនរបស់អ្នក

```python
class PickupTask:
    def __init__(self, offer_id, volunteer, portions):
        self.offer_id = offer_id
        self.volunteer = volunteer
        self.portions = portions

    def describe(self):
        return f"{self.offer_id}: {self.volunteer} collects {self.portions} portions"
```

- ⭐⭐ បន្ថែម: បន្ថែម `status` និង `move_to` ជាមួយ `waiting → on_the_way → done`។

⚖️ ល្អ ឬមានហានិភ័យ?

Version A (ពីជំនួយការ AI)

```
def update_status(offer, typed):
    offer.status = typed.strip().lower()

def mark_delivered(offer):
    offer.status = "delivered"

def fix_mistake(offer):
    offer.status = "new"          # reset
```

"ប៊ូតុងសម្រាប់អ្នកស្ទាត់ចិត្ត និងវិធីកែកំហុស។"

Version B

```
def update_status(offer, new_status, is_allowed):
    old = offer.status
    if offer.move_to(new_status, is_allowed):
        return "done: " + old + " -> " + new_status
    return "refused: " + old + " -> " + new_status
```

ទ្វារតែមួយ វិធានតែមួយ ចម្លើយច្បាស់លាស់។

អ្វីដែល Version A ធ្វើចំពោះសំណើមួយ

ជំហាន	status ក្លែង	នរណាឯងគ្រោះ
ចុចម្តងលើ "delivered"	delivered	គ្មាននរណាទៅយក curry ទេ: វាខូច ហើយផលប្លុកដែលបានប្រគល់មិនពិត
"កែកំហុស"	new	អ្នកស្ម័គ្រចិត្តបើកឡានទៅយកអាហារដែលអស់ហើយ; ចំណែកត្រូវបានរាប់ពីរដង
អ្នកស្ម័គ្រចិត្តវាយ `Deliverd`	deliverd	សំណើបាត់ពីគ្រប់បញ្ជី និងគ្រប់ផលប្លុក

- គ្មានចម្លើយត្រូវបាន return ហើយគ្មានវិធានតែមួយសម្រាប់អាន ឬ test ទេ។

> Version B ល្អជាង ប៉ុន្តែ Python នៅតែអនុញ្ញាត `offer.status = ...` នៅកន្លែងផ្សេង។ ទ្វារតែមួយ ហើយពិនិត្យ code។

គោលការណ៍ទី 6 · ធ្វើឱ្យស្ថានភាពខុស កើតឡើងបានពិបាក

1. **អនុញ្ញាត តែ** ការប្តូរដែលសមហេតុផល។
2. រក្សាវិធាន **នៅជាប់នឹងទិន្នន័យ**: `move_to` ស្ងួរមុនពេលវាប្តូរ។
3. ប្តូរ status **តែ** តាមរយៈទ្វារតែមួយនោះ។

> Python មិនអាចធ្វើឱ្យស្ថានភាពខុស មិនអាចកើតឡើងបានទេ ដូច្នេះក្រុមត្រូវព្រៀងលើទ្វារតែមួយ ហើយពិនិត្យ code រកទ្វារផ្សេងទៀត។

អ្នកបង្កើត: `can_move(old_status, new_status)`

```
ALLOWED_MOVES = {
    "new": ["approved", "rejected"],
    "approved": ["picked_up", "cancelled"],
    # TODO 1: picked_up, delivered, rejected, cancelled
}
# TODO 2: បើ not REQUIRE_APPROVAL, new អាចទៅផ្ទាល់ picked_up

def can_move(old_status, new_status):
    # TODO 3: old_status មិនស្គាល់ -> False
    # TODO 4: True តែពេល new_status នៅក្នុង list របស់វា
    return False
```

ការសម្រេចចិត្តមួយ កន្លែងពីរ

```
REQUIRE_APPROVAL = True
MY_SETTINGS = {"REQUIRE_APPROVAL": REQUIRE_APPROVAL}

if not REQUIRE_APPROVAL:                          # បង្កើត dict ពី setting
    ALLOWED_MOVES["new"].append("picked_up")

fr.check("can_move", can_move, MY_SETTINGS)
```

- ការពិនិត្យស្នូរ `can_move("new", "picked_up")`: `False` ពេលមានការអនុម័ត `True` ពេលគ្មាន។
- បើ setting និង dictionary មិនស៊ីគ្នា ប្រព័ន្ធនិយាយមួយបែប តែធ្វើមួយបែបទៀត។

ភ្ជាប់វាចូលប្រព័ន្ធ

```
my_system = fr.System(MY_SETTINGS, hooks={"can_move": can_move})
my_system.run_day()
ok, message = my_system.move("F004", "delivered")    # ប្រគល់មុនពេលទៅយក
print(ok, message)     # False F004 មិនអាចប្តូរពី បានអនុម័ត ទៅ បានប្រគល់ បានទេ។
```

- ក្នុង notebook: 🚫 ✅ ✅ 🚫 🚫 សម្រាប់ការសាកល្បងប្រាំដង។
- 🚫 នីមួយៗ គឺជាស្ថានភាពខុសដែលវិធានរបស់អ្នកបានរារាំង។ record មិនបានប្រែប្រួលទេ។

🧭 ការសម្រេចចិត្តរបស់អ្នក · អនុម័តមុនពេលទៅយក?

- `REQUIRE_APPROVAL = True`: អ្នកសម្របសម្រួលពិនិត្យថា អង្គការដៃគូអាចទទួលអាហារបាន។
- `REQUIRE_APPROVAL = False`: អាហារបន្ទាន់អាចទៅយកបានភ្លាមៗ។
  - ប្រព័ន្ធបង្ហាញការព្រមាន ⚠️ **Setting មានហានិភ័យ**។
  - កំណត់ត្រាការសម្រេចចិត្តរបស់អ្នក **ត្រូវតែ** ផ្តល់ហេតុផលសម្រាប់វា។

> តើនរណាត្រូវបានការពារដោយការរង់ចាំ ហើយនរណាបាត់បង់អាហារដោយសារវា?

កិច្ចការសប្តាហ៍នេះ

- ⭐ **មូលដ្ឋាន:** P1–P4 ការពិភាក្សា ⚖️ `can_move` ដែលមាន ✅ ទាំងអស់ ភ្ជាប់ចូលប្រព័ន្ធ កំណត់ត្រាការសម្រេចចិត្ត។
- ⭐⭐ **បន្ថែម:** lifecycle សម្រាប់ `PickupTask`; `next_moves(status)` ដើម្បីឱ្យអ្នកក្រង់បង្ហាញតែចំតុងដែលបានអនុញ្ញាត។
- 🔎 **កិច្ចការស្រាវជ្រាវទី 5 (5%):** ការរក្សាទិន្នន័យឱ្យមានសុវត្ថិភាព: backup, audit log, transaction។ update ពីរដែលត្រូវតែកើតទ្បើងជាមួយគ្នា; អ្វីដែលអាចខុសនៅចន្លោះវា; របៀបពិនិត្យមុន និងក្រោយ។ ចែករំលែកវានៅពេលចាប់ផ្តើមថ្នាក់សប្តាហ៍ទី 7។

សប្តាហ៍ក្រោយ · ការរក្សាទុក និងប្រវត្តិ

កន្ទុរនេះ អ្វីៗទាំងអស់បាត់ទៅវិញ ពេល notebook បិទ។ អ្នកនឹង **រក្សាទុក** record ដោយសុវត្ថិភាព ហើយរក្សា **ប្រវត្តិ** នៃរាល់ការប្តូរ។